

Lecture 21

Registers and Ripple Counters

Registers and parallel load · shift registers · the universal shift register · serial adder · ripple counters

1	0	1	1
---	---	---	---

0	1	0	1
---	---	---	---

shift right, serial in 0

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 6, §6.1–6.3

Today's roadmap

1

§6.1

Registers

Store and load a word

2

§6.2

Shift registers

Move bits each clock

3

§6.2

Serial adder

One full adder, n clocks

4

§6.3

Ripple counters

Simple, slow, glitchy

By the end of class you can...

- Add parallel load to a register without gating the clock
- Trace SIPO, PISO and universal shift registers
- Trace a serial adder bit by bit
- Explain how a ripple counter counts and why it glitches
- Compute settling time and frequency division

Agenda for today's session

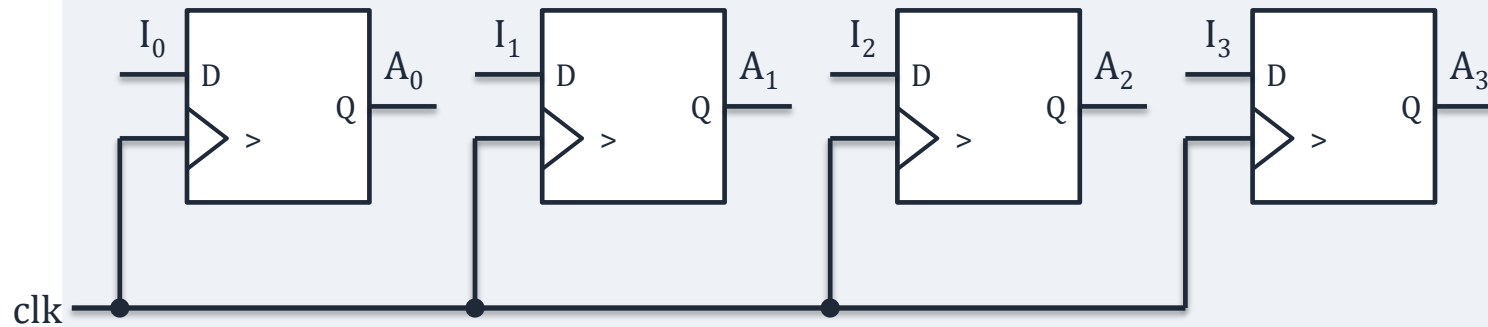
Intro	Registers	Shift	Break	Serial	Ripple	Wrap	
0:00	0:05	0:25	0:50	1:00	1:15	1:40	1:50

Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome; Homework 9 collected	—
0:05 – 0:25	Registers and parallel load	Example 1
0:25 – 0:50	Shift registers; universal register	Examples 2, 3
0:50 – 1:00	Break	
1:00 – 1:15	Serial transfer and the serial adder	Example 4
1:15 – 1:40	Ripple counters and their timing	Example 5 · Your turn
1:40 – 1:50	Verilog; concept check; wrap-up	—

Registers

§6.1

4-bit register: one D flip-flop per bit



All flip-flops share one clock: the word is stored at the same edge.

Most registers add an asynchronous clear to start at zero.

A register

n flip-flops holding an n -bit word.

Registers store; counters sequence; together with combinational logic they are every digital system.

Problem

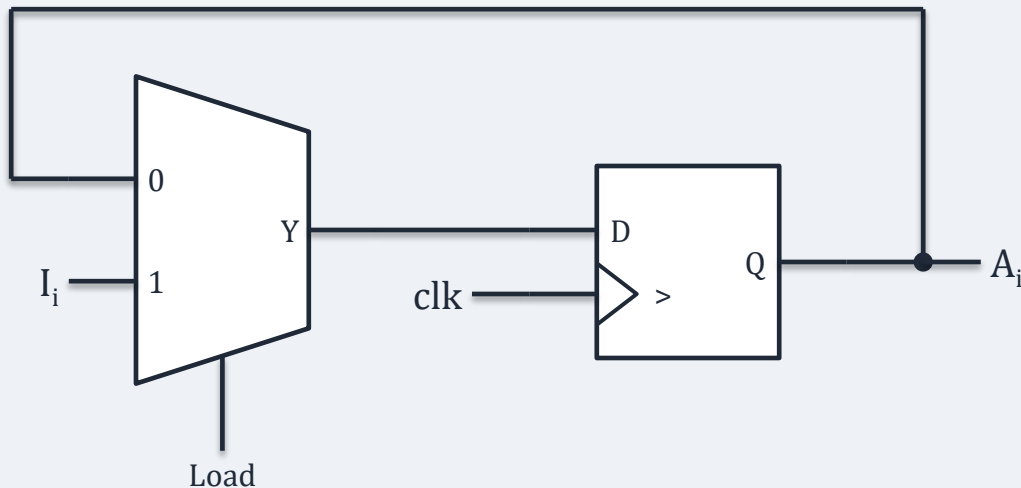
As drawn, the register loads on every clock. How do we make it hold?

Example 1 — Parallel load

§6.1

EXAMPLE

Make each bit load I_i when $Load = 1$ and hold when $Load = 0$, without touching the clock.



One stage; repeat for every bit, sharing Load and clk.

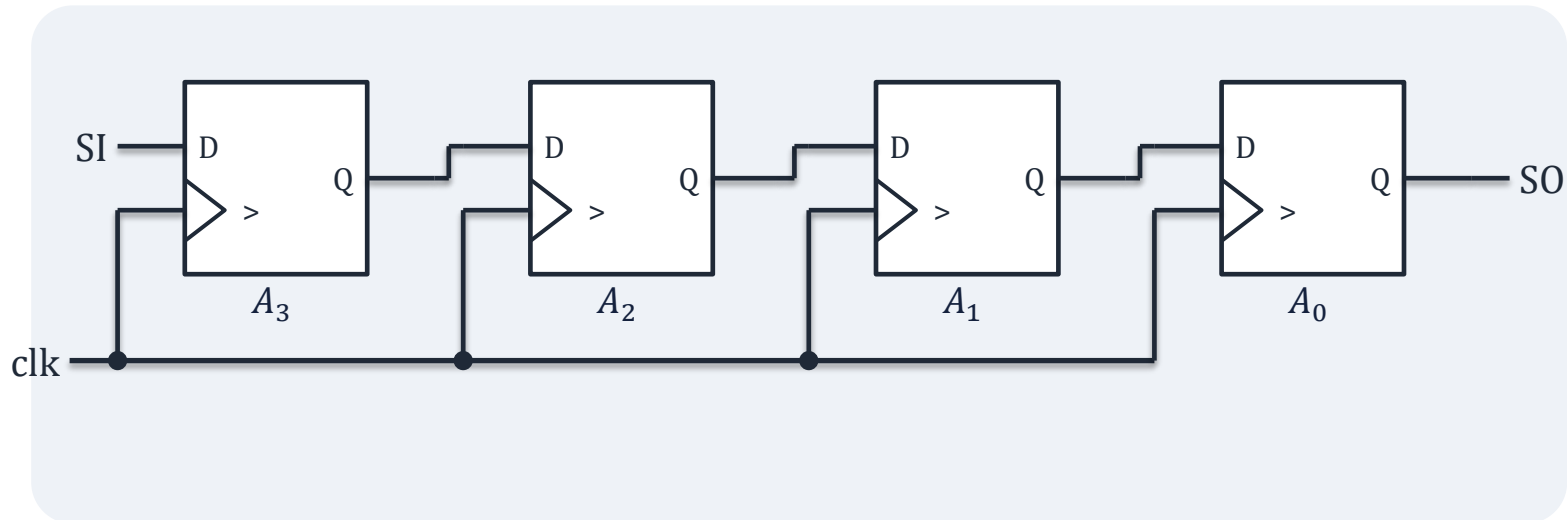
- 1 Each bit: a 2-to-1 MUX in front of D , selected by $Load$
- 2 $D_i = Load' A_i + Load I_i$
- 3 $Load = 0$: $D_i = A_i$: the flip-flop reloads itself = hold
- 4 $Load = 1$: $D_i = I_i$: new data enters on the next edge
- 5 The clock runs every cycle either way
- 6 Gating clk with $Load$ adds skew and glitches: never

Example 2 — A shift register

§6.2

EXAMPLE

A 4-bit shift-right register starts at 0000. The serial input is 1, 0, 1, 1 on four clocks. Trace it.



Clock	SI	$A_3A_2A_1A_0$
0	—	0000
1	1	1000
2	0	0100
3	1	1010
4	1	1101

- 1 Each clock: every bit moves one place right
- 2 SI enters A_3 ; A_0 leaves on SO
- 3 Clock 1, SI = 1: 0000 → 1000
- 4 Clock 2, SI = 0: 1000 → 0100
- 5 Clock 3, SI = 1: 0100 → 1010
- 6 Clock 4, SI = 1: 1010 → **1101**
- 7 Serial in, parallel out (SIPO); the first bit is in A_0

Example 3 — Universal shift register

§6.2

EXAMPLE

Start at 1001. Right-shift input 0, left-shift input 1, parallel input 1100. Apply $s_1s_0 = 01, 10, 10, 11, 00$.

$s_1 s_0$	Operation
0 0	hold
0 1	shift right
1 0	shift left
1 1	parallel load

Each stage has a 4-to-1 MUX on its D input; s_1s_0 selects the same source for every stage.

Clock	s_1s_0	$A_3A_2A_1A_0$
0	—	1001
1	01	0100
2	10	1001
3	10	0011
4	11	1100
5	00	1100

- 1 Modes: 00 hold, 01 right, 10 left, 11 load
- 2 01: right, 0 enters A_3 : 1001 $\rightarrow$ 0100
- 3 10: left, 1 enters A_0 : 0100 $\rightarrow$ 1001
- 4 10: left again: 1001 $\rightarrow$ 0011
- 5 11: load I : 0011 $\rightarrow$ 1100
- 6 00: hold: 1100 $\rightarrow$ **1100**

Break

10 minutes — we resume at the 1:00 mark

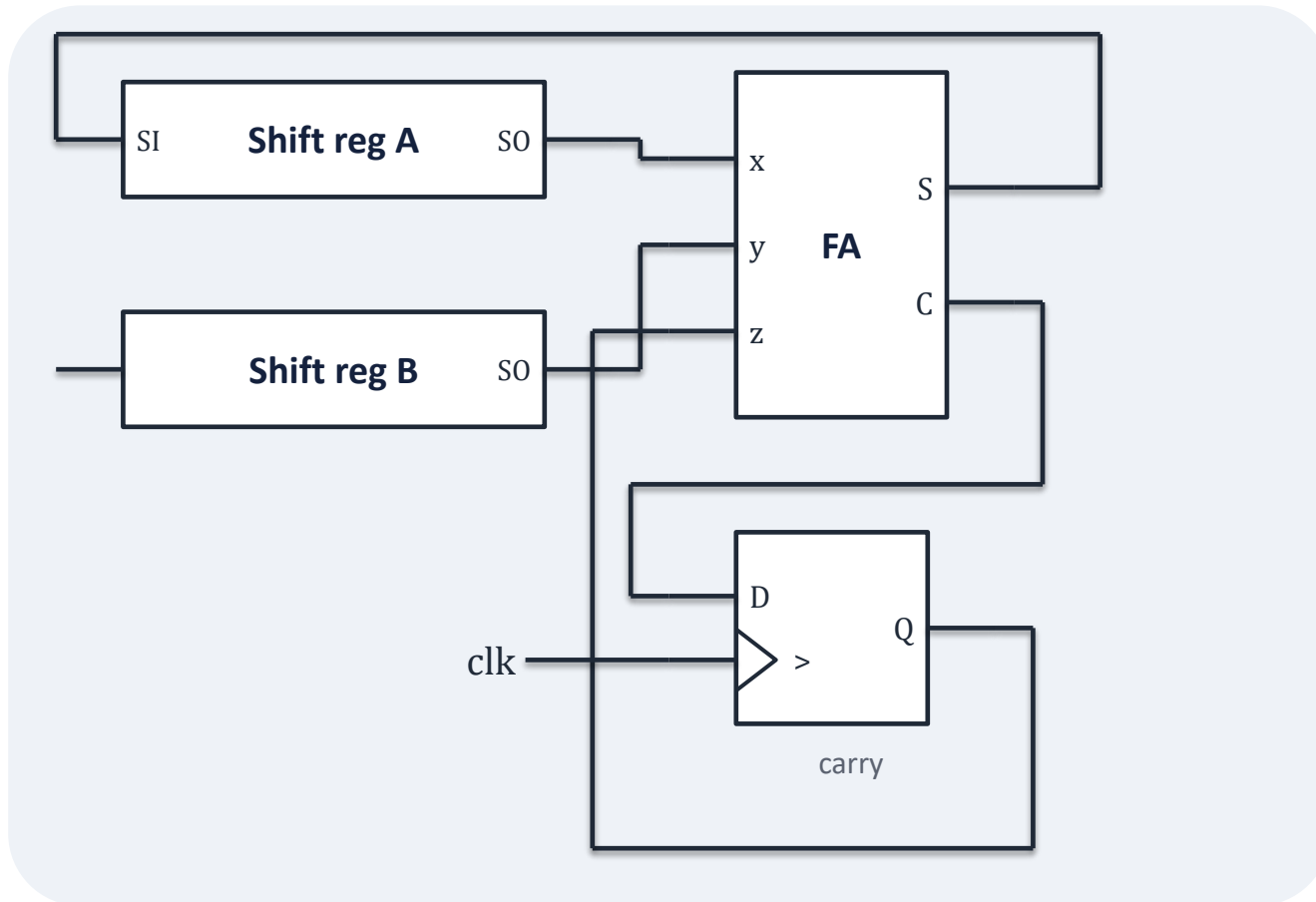
Next: the serial adder and ripple counters

Example 4 — The serial adder

§6.2

EXAMPLE

Add $A = 1101$ and $B = 0111$ one bit per clock with one full adder and a carry flip-flop. The sum goes back into A.



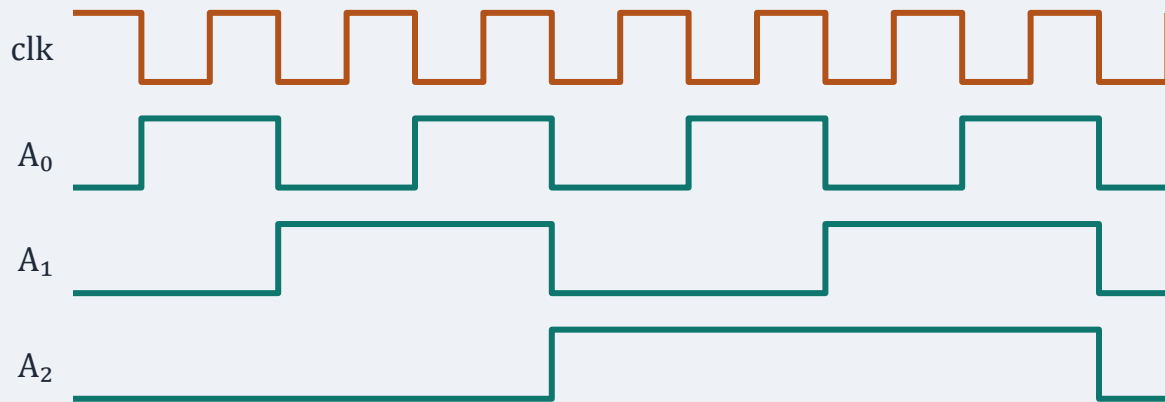
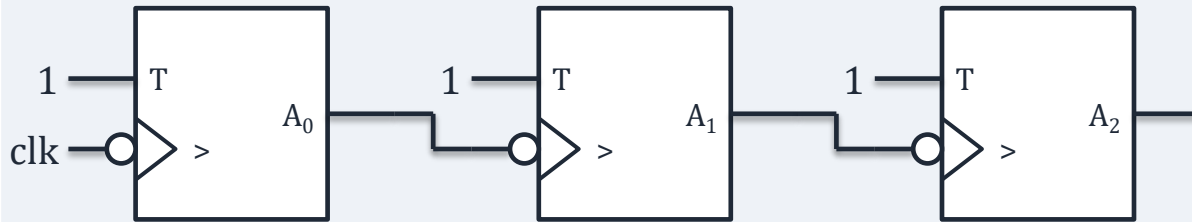
Clock	x	y	z=Q	S	C
1	1	1	0	0	1
2	0	1	1	0	1
3	1	1	1	1	1
4	1	0	1	0	1

- 1 Clear the carry flip-flop: $z = 0$ before clock 1
- 2 Each clock: add x, y, z ; $S \rightarrow A$, $C \rightarrow$ flip-flop
- 3 Sum bits 0, 0, 1, 0: **A = 0100**, carry 1
- 4 $13 + 7 = 20 = 1\ 0100$ ✓: n clocks, one adder

The ripple counter

§6.3

3-bit ripple up-counter (negative edge)



How it counts

Only the first stage sees the clock; each stage toggles when the one before it falls from 1 to 0.

Each stage divides the frequency by 2: $A_0 = f/2$, $A_1 = f/4$, $A_2 = f/8$.

Down-counter

Clock each stage from the previous Q' instead.

Cheap: no gates at all.

Example 5 — Why ripple counters glitch

§6.3

EXAMPLE

A 3-bit ripple counter goes from 011 to 100. Each stage has $t_{pd} = 10$ ns. What do the outputs show on the way?

Time	Event	$A_2A_1A_0$	Valid?
0	clock falls	011	yes (old)
10 ns	$A_0: 1 \rightarrow 0$	010	no
20 ns	$A_1: 1 \rightarrow 0$	000	no
30 ns	$A_2: 0 \rightarrow 1$	100	yes (new)

- 1 Clock falls: only A_0 is clocked
- 2 10 ns: $A_0: 1 \rightarrow 0$; that edge clocks A_1 : shows 010
- 3 20 ns: $A_1: 1 \rightarrow 0$; that edge clocks A_2 : shows 000
- 4 30 ns: $A_2: 0 \rightarrow 1$: shows 100. Settled after $3 \times 10 = 30$ ns
- 5 Two false counts (010, 000) appear on the way
- 6 A decoder on the outputs sees them briefly: glitches
- 7 Fix: synchronous counters (Lecture 22)

Your turn — Registers and counters

§6.2

YOUR TURN

≈ 10 min

- (a) A shift-right register holds 1011. The serial input (into the MSB) is 0, 1, 1, 0. Give the contents after each clock.
- (b) A universal shift register holds 0110. With $s_1s_0 = 10$ and left serial input 1, what is the next state?
- (c) A 4-bit ripple counter has 15 ns stages. Worst-case settling time and maximum count rate?
- (d) How many stages divide 1 MHz to below 1 kHz?

Your turn — Solution

§6.2

SOLUTION

- 1 (a) Each clock: shift right; the serial bit enters the MSB
- 2 Clock 1, SI = 0: 1011 → 0101
- 3 Clock 2, SI = 1: → 1010; clock 3, SI = 1: → 1101
- 4 Clock 4, SI = 0: → **0110**
- 5 (b) $s_1s_0 = 10$: shift left; 1 enters at the LSB
- 6 0110 → **1101**
- 7 (c) Worst case 0111 → 1000: all 4 stages change in turn
- 8 $4 \times 15 = \mathbf{60\ ns}$; $f_{max} = 1/60\ ns = \mathbf{16.7\ MHz}$
- 9 (d) Each stage divides by 2: $2^{10} = 1024 \geq 1000 \rightarrow \mathbf{10\ stages}$
- 10 Output $1\ MHz/1024 = 976.6\ Hz$

Registers in Verilog

§5.6

```
module shreg4 (input clk, load, si,
               input  [3:0] I,
               output reg [3:0] A);
    always @(posedge clk)
        if (load) A <= I;
        else      A <= {si, A[3:1]};
endmodule

module count4 (input clk, rst,
               output reg [3:0] Q);
    always @(posedge clk)
        if (rst) Q <= 4'b0000;
        else     Q <= Q + 1;
endmodule
```

Notes

always @(posedge clk) describes flip-flops.

Use nonblocking <= so all bits update together.

{si, A[3:1]} concatenates: a right shift with si entering the MSB.

count4 synthesizes a synchronous counter, not a ripple one.

Concept check

1 Why should a register never gate its clock to hold data?

2 What does a SIPO shift register do?

3 Why does a ripple counter show false counts?

4 How many ripple stages divide a clock by 256?

Key takeaways

Load

$$D_i = Load' A_i + Load I_i$$

Shift right

SI $\rightarrow$ MSB, LSB out

Universal

hold, right, left, load

Serial adder

1 FA + carry FF, n clocks

Ripple

each stage $\div 2$

Settling

$n \cdot t_{pd}$, false counts

Registers hold words; counters step through them.

Homework #10 due at the start of Lecture 22

Next: synchronous counters